The Feedback Vertex Set Problem: Theory, Algorithms, and Applications

Project Checkpoint Presentation, CSE 462

2005090 - Tawkir Aziz Rahman
2005074 - Dipanta Kumar Roy Nobo
2005091 - Waseem Mustak Zisan
2005104 - Hasin Arafat
2005109 - Noushin Tabassum Aoishy
2005068 - Suman Hossain

Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology

January 26, 2026

Outline

Problem Definition

Overview

- Define the Feedback Vertex Set (FVS) problem and formal statement.
- Provide visual examples and intuition (cycles vs. forests).
- Highlight key properties: NP-completeness and FPT results.

What is the Feedback Vertex Set Problem?

Definition (Feedback Vertex Set)

Given an undirected graph $G = (V, E)$ and an integer k, find a set $S \subseteq V$ with $|S| \leq k$ such that $G - S$ is **acyclic** (a forest).

Formal Problem:

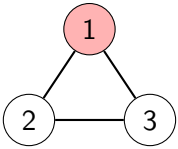
- **Input:** Graph $G = (V, E)$, integer k
- **Question:** $\exists S \subseteq V$, $|S| \leq k$ such that $G - S$ has no cycles?
- **Optimization:** Find minimum $|S|$

Intuition:

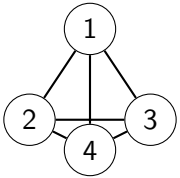
- Break all cycles by removing vertices
- Every cycle must contain ≥ 1 vertex from S
- Minimize disruption (minimize $|S|$)

Visual Examples

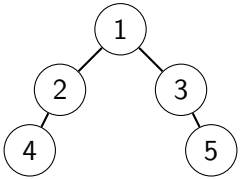
Triangle (3-cycle)



Complete Graph K_4



Tree

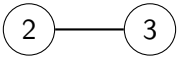


Key Property

A set S is a feedback vertex set if and only if removing S leaves no cycles.

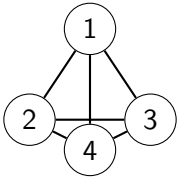
Visual Examples

Triangle (3-cycle)

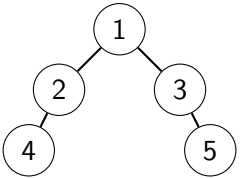


FVS= $\{1\}$, Size $= 1$

Complete Graph K_4



Tree

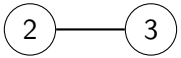


Key Property

A set S is a feedback vertex set if and only if removing S leaves no cycles.

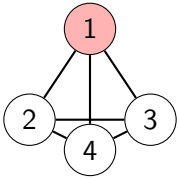
Visual Examples

Triangle (3-cycle)

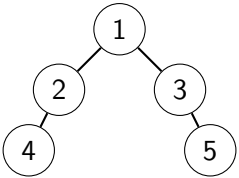


FVS= $\{1\}$, Size $= 1$

Complete Graph K_4



Tree

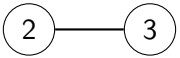


Key Property

A set S is a feedback vertex set if and only if removing S leaves no cycles.

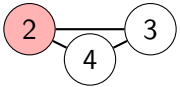
Visual Examples

Triangle (3-cycle)

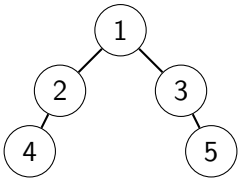


FVS= $\{1\}$, Size $= 1$

Complete Graph K_4



Tree

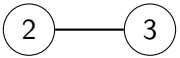


Key Property

A set S is a feedback vertex set if and only if removing S leaves no cycles.

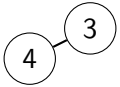
Visual Examples

Triangle (3-cycle)



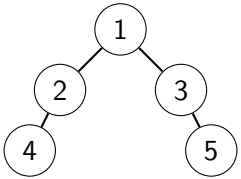
FVS= {1}, Size = 1

Complete Graph K_4



FVS= {1,2}, Size = 2

Tree

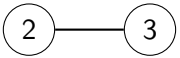


Key Property

A set S is a feedback vertex set if and only if removing S leaves no cycles.

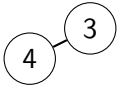
Visual Examples

Triangle (3-cycle)



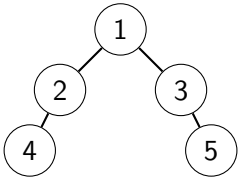
FVS= {1}, Size = 1

Complete Graph K_4



FVS= {1,2}, Size = 2

Tree

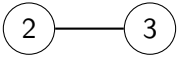


Key Property

A set S is a feedback vertex set if and only if removing S leaves no cycles.

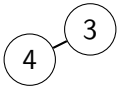
Visual Examples

Triangle (3-cycle)



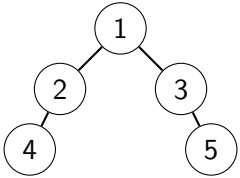
FVS= {1}, Size = 1

Complete Graph K_4



FVS= {1,2}, Size = 2

Tree



FVS= $\emptyset$, Size = 0

Key Property

A set S is a feedback vertex set if and only if removing S leaves no cycles.

Important Properties

1. **NP-Completeness:** FVS is NP-complete (shown via reductions from problems such as Vertex Cover).
   A problem is NP-complete if it is in NP and every problem in NP can be reduced to it in polynomial time.

Important Properties

1. **NP-Completeness:** FVS is NP-complete (shown via reductions from problems such as Vertex Cover).
   A problem is NP-complete if it is in NP and every problem in NP can be reduced to it in polynomial time.
2. **Parameterized Complexity:** Fixed-Parameter Tractable (FPT).

Important Properties

1. **NP-Completeness:** FVS is NP-complete (shown via reductions from problems such as Vertex Cover).
   A problem is NP-complete if it is in NP and every problem in NP can be reduced to it in polynomial time.
2. **Parameterized Complexity:** Fixed-Parameter Tractable (FPT).
   - Solvable in $O(f(k) \cdot \mathrm{poly}(n))$ time.
   - One of the best known FPT algorithms runs in time $O(c^k \cdot \mathrm{poly}(n))$ for some $c < 4$.

Important Properties

1. **NP-Completeness:** FVS is NP-complete (shown via reductions from problems such as Vertex Cover).
   A problem is NP-complete if it is in NP and every problem in NP can be reduced to it in polynomial time.
2. **Parameterized Complexity:** Fixed-Parameter Tractable (FPT).
   - Solvable in $O(f(k) \cdot \mathrm{poly}(n))$ time.
   - One of the best known FPT algorithms runs in time $O(c^k \cdot \mathrm{poly}(n))$ for some $c < 4$.
3. **Bounds:**
   - A commonly used heuristic lower bound is $|E| - |V| + c$ (where $c =$ number of connected components).
   - Upper bound example: $|S| \leq |V| - 1$.

Important Properties

1. **NP-Completeness:** FVS is NP-complete (shown via reductions from problems such as Vertex Cover).
   A problem is NP-complete if it is in NP and every problem in NP can be reduced to it in polynomial time.
2. **Parameterized Complexity:** Fixed-Parameter Tractable (FPT).
   - Solvable in $O(f(k) \cdot \mathrm{poly}(n))$ time.
   - One of the best known FPT algorithms runs in time $O(c^k \cdot \mathrm{poly}(n))$ for some $c < 4$.
3. **Bounds:**
   - A commonly used heuristic lower bound is $|E| - |V| + c$ (where $c =$ number of connected components).
   - Upper bound example: $|S| \leq |V| - 1$.
4. **Special Cases:**
   - Trees/Forests: $|S| = 0$.
   - Planar graphs: Better approximation algorithms exist.

Reductions: Why They Matter

Purpose of Reductions

- Prove FVS is NP-hard by reducing from known NP-complete problems
- Show relationships between different hard problems
- Transfer algorithmic techniques between problem domains

Key Reductions:

- Vertex Cover $\rightarrow$ FVS
- FVS$\rightarrow$ Vertex Cover
- 3-SAT $\rightarrow$ FVS
- FVS$\rightarrow$ Independent Set problem

Complexity Implications:

- FVS is NP-complete
- No polynomial algorithm unless P=NP
- Same approximation barriers as Vertex Cover

Reduction 1: Vertex Cover $\rightarrow$ FVS

Vertex Cover (VC) Problem

Find smallest $S \subseteq V$ such that every edge has at least one endpoint in S.

Construction: Given VC instance (G, k)

1. For each edge $e = (u, v) \in E(G)$:
   - Add new vertex w_e
   - Form triangle: edges (u, v), (u, w_e), (v, w_e)
2. Keep same parameter $k' = k$

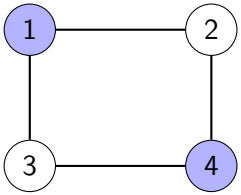
Theorem

G has VC of size $\leq k \Leftrightarrow G'$ has FVS of size $\leq k$

Time: $O(|E|)$ (polynomial)

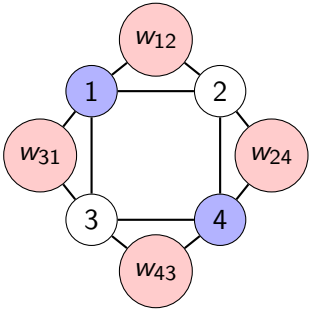
VC $\rightarrow$ FVS: Example & Proof

Original Graph G:



Minimum Vertex Cover: $\{1, 4\}$ (size=2)
Covers edges: (1,2), (2,4), (4,3), (3,1)

Transformed Graph G':



FVS: $\{1, 4\}$ breaks all 4 triangles
Each triangle = one original edge

Reduction 2: $\mathrm{FVS} \rightarrow$ Vertex Cover

Reverse Direction

Shows Vertex Cover and FVS are polynomially equivalent

Construction: Given FVS instance (G, k)

1. Replace each vertex $v \in V$ with triangle $\{v_1, v_2, v_3\}$
2. For each edge $(u, v) \in E$, connect triangles with cross edges
3. Set $k' = 2|V| + k$

Theorem

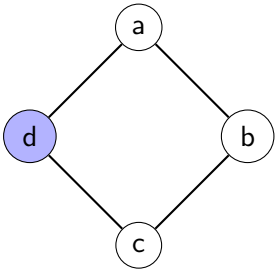
G has FVS of size $\leq k \Leftrightarrow G'$ has VC of size $\leq 2|V| + k$

Key Insight:

- Each triangle needs ≥ 2 vertices in VC
- Triangles with 3 vertices in VC $\leftrightarrow$ vertices in FVS

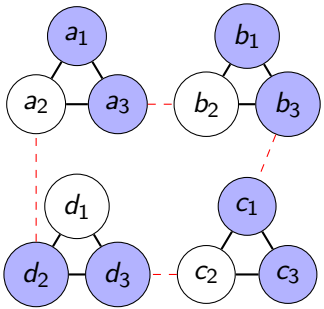
FVS→ VC: Visual Example

Original Graph G:



FVS: $\{d\}$ (size=1)
Removing d breaks the 4-cycle

Transformed Graph G' (simplified view):



Dashed = original edges , Each triangle = one vertex in G

Vertex Cover in G': is also a FVS

- Each triangle needs ≥ 2 vertices in VC

Other Important Reductions

3-SAT $\rightarrow$ FVS(Standard NP-hardness proof)

- **Variable gadget:** Triangle $\{x, \neg x, t_x\}$
- **Clause gadget:** Cycle connecting 3 literal vertices
- **Reduction:** ϕ satisfiable $\leftrightarrow$ G_ϕ has FVS of size n
- **Time:** $O(n + m)$ (linear)

Quick Examples of Other Reductions

- **Independent Set $\rightarrow$ FVS:** Via complement graph transformation
- **Dominating Set $\rightarrow$ FVS:** On directed line graphs

Reduction Complexity Summary

Reduction	Time	Preserves Solution Size?
Vertex Cover $\rightarrow$ FVS	$O(\|E\|)$	Yes (exactly)
FVS$\rightarrow$ Vertex Cover	$O(\|V\| + \|E\|)$	Linear: $k' = 2\|V\| + k$
3-SAT $\rightarrow$ FVS	$O(n + m)$	Linear: $k' = n$ (#variables)
FVS$\rightarrow$ Independent Set	$O(\|V\| + \|E\|)$	Within factor 2

Key Takeaways

1. Vertex Cover and FVS are **polynomially equivalent**
2. Both are NP-complete (reducible from 3-SAT)
3. Reductions preserve approximation hardness
4. Algorithmic techniques transfer between problems

Implications for Algorithms

What We Gain:

- FVS inherits Vertex Cover's:
  - 2-approximation algorithm
  - Kernelization results
  - FPT algorithms (parameter k)
  - Branching techniques
- Can use SAT solvers via 3-SAT reduction

Limitations:

- No polynomial algorithm (unless P=NP)
- Hard to approximate better than 2 (under UGC)
- W[1]-hard for some parameters

The Big Picture

Reductions create a network of equivalent hard problems. Solving one efficiently would solve them all (unlikely under $\mathrm{P} \neq \mathrm{NP}$).

Algorithm Survey

Overview

- Classify algorithms: Exact, Approximation, Randomized, Heuristic, Metaheuristic.
- Present representative methods and their time/quality trade-offs.
- Highlight algorithms selected for implementation and evaluation.

Five Major Categories of Solution Approaches

1. Exact Exponential

- Naive Brute Force
- DP on Subsets
- Iterative Compression
- Bounded Search Tree
- Current Best FPT
- Inclusion-Exclusion DP

2. Approximation

- 2-Approximation (VC-based)
- Log-Approximation
- LP Rounding
- Primal-Dual

3. Randomized

- Random Sampling
- Randomized Rounding
- Color Coding

4. Heuristic

- Greedy Vertex Selection
- Cycle-Based Greedy
- Maximum Degree Heuristic
- Core-Periphery Decomposition
- Reduction Rules + Greedy

5. Metaheuristic

- Genetic Algorithm
- Simulated Annealing
- Ant Colony Optimization
- Tabu Search
- Particle Swarm Optimization

Category 1: Exact Exponential Algorithms

Algorithm	Description	Complexity	Solution Quality
Naive Brute Force	Enumerate all 2^n subsets and check if each forms a valid FVS using cycle detection	$O(2^n \cdot n^3)$	Optimal
DP on Subsets	Dynamic programming to avoid recomputing subproblems; for each subset, determine if it's a valid FVS	$O(2^n \cdot n^2)$	Optimal
Iterative Compression	FPT algorithm that assumes solution of size $k+1$ and compresses to size k (Dehne et al., 2004)	$O(5^k \cdot kn^2)$	Optimal
Bounded Search Tree	Use branching rules to create search tree with depth $\leq k$; improved branching factor (Chen et al., 2006)	$O(4^k \cdot n^2)$	Optimal
Current Best FPT	State-of-art using crown decomposition, sunflower lemma, refined branching (Kociumaka & Pilipczuk, 2010)	$O(3.619^k \cdot n^4)$	Optimal
Inclusion-Exclusion DP	Uses inclusion-exclusion principle with DP for counting; fast for very small graphs ($n \leq 25$)	$O(2^n \cdot n)$	Optimal

Key Insight: FPT algorithms practical for $k \leq 50$; exponential in n feasible only for $n \leq 20$

Category 2: Approximation Algorithms

Algorithm	Description	Complexity	Solution Quality
2-Approximation (VC-based)	Reduce FVS to Vertex Cover; iteratively find cycles and remove both endpoints of any edge (Bafna et al., 1999)	$O(n^2)$	2-approx
Log-Approximation	For tournament graphs using hitting set techniques; specific to directed complete graphs	$O(n^2 \log n)$	$O(\log n)$-approx
LP Rounding	Formulate as ILP, solve LP relaxation, then round fractional solution (deterministic or randomized)	$O(n^{3.5})$	2-approx
Primal-Dual	Maintain primal and dual solutions simultaneously; combinatorial approach without LP solver	$O(n^2)$	2-approx
Local Ratio	Decompose weight function and apply local decisions; works well with weighted vertices	$O(n^2)$	2-approx

Open Question: Does $(2 - \varepsilon)$-approximation exist? Under UGC, 2 might be optimal.

Category 3: Randomized Algorithms

Algorithm	Description	Complexity	Solution Quality
Random Sampling	Randomly sample vertices in random order; if vertex in cycle, include it in FVS; Monte Carlo approach	$O(n^2)$ per trial	Expected $O(\log n)$-approx
Randomized Rounding	Solve LP relaxation, then randomly include vertex v with probability x_v^*; LP-based randomization	$O(n^3)$	Expected 2-approx
Color Coding	Randomly color vertices and use DP to find colorful solutions; for bounded treewidth graphs	$O(2^{O(k)} \cdot n)$	Optimal w.h.p.

Key Advantages

- Probabilistic guarantees with high probability (w.h.p.)
- Can be derandomized using method of conditional expectations
- Excellent for sparse graphs and bounded treewidth structures

Category 4: Heuristic Algorithms

Algorithm	Description	Complexity	Solution Quality
Greedy Vertex Selection	Repeatedly remove vertex with maximum degree; simple and fast; uses priority queue	$O(n^2 \log n)$	No guarantee
Cycle-Based Greedy	Find cycles using DFS/BFS and select vertex appearing in most cycles; iterative improvement	$O(n^3)$	No guarantee
Maximum Degree Heuristic	Like max degree but with tie-breaking: clustering coefficient, betweenness centrality	$O(n^2 \log n)$	No guarantee
Core-Periphery Decomposition	Compute k-core decomposition; focus on highly connected core vertices	$O(n + m)$	No guarantee
Reduction Rules + Greedy	Apply polynomial-time reduction rules first (degree 0,1,2 vertices, self-loops) then greedy	$O(n^2)$	No guarantee

Practical Value: Often reduce graph size by 50-90% before greedy phase; excellent performance on real-world networks

Category 5: Metaheuristic Algorithms

Algorithm	Description	Complexity	Solution Quality
Genetic Algorithm	Evolve population using selection, crossover, mutation; binary encoding or permutation-based	$O(g \cdot p \cdot n^2)$	No guarantee
Simulated Annealing	Probabilistic technique accepting worse solutions with decreasing probability; geometric cooling schedule	$O(i \cdot n^2)$	No guarantee
Ant Colony Optimization	Simulate ant foraging using pheromone trails; balance exploration vs exploitation	$O(\text{ants} \cdot i \cdot n^2)$	No guarantee
Tabu Search	Local search with memory (tabu list) to avoid cycling; aspiration criterion for best solutions	$O(i \cdot n^2)$	No guarantee
Particle Swarm Optimization	Swarm intelligence inspired by bird flocking; particles explore solution space collaboratively	$O(\text{particles} \cdot i \cdot n^2)$	No guarantee

Legend: g = generations, p = population size, i = iterations

Implementation Plan

Overview

- Describe selected algorithms to implement (Iterative Compression, Genetic Algorithm).
- Outline implementation details and complexity considerations.

Algorithm Selection Criteria

Essential Criteria

1. **Implementability**
   - Feasible in 6-week timeframe
   - Well-documented in literature
   - Reasonable debugging complexity
2. **Research Potential**
   - Novel comparisons possible
   - Room for improvements
   - Publishable results
3. **Complementary Nature**
   - Different problem sizes
   - Different paradigms
   - Interesting comparisons

Additional Criteria

4. **Practical Relevance**
   - Real-world applications
   - Industry interest
   - Addresses practical needs
5. **Educational Value**
   - Learn important techniques
   - Transferable skills
   - Deep algorithmic insights

Goal

Select algorithms that maximize research impact while ensuring feasibility

Selected Algorithms for Implementation

Three-Algorithm Approach (Maximum Research Impact)

We will implement three complementary algorithms covering the entire solution spectrum:

1. **Iterative Compression** (Exact FPT Algorithm)
   - Guarantees optimal solution for small-medium k
   - Time: $O(5^k \cdot kn^2)$ — FPT breakthrough (2004)
   - Research: Hybrid approaches, parallelization, adaptive branching
2. **Kernelization + Bounded Search Tree** (Enhanced Exact)
   - Preprocessing with reduction rules + exact search
   - Kernel reduction: 50-90% for many graph types
   - Research: New reduction rules, smart branching strategies
3. **Memetic Algorithm** (Advanced GA + Local Search)
   - Handles large instances (n $>$ 1000)
   - Combines population evolution with local optimization
   - Research: Problem-specific operators, ML integration, hybrid exact-metaheuristic

Algorithm 1: Iterative Compression (IC)

Core Concept

Given FVS of size $k+1$, compress it to size k through smart enumeration

Theoretical Significance:

- Most important FPT technique
- Published in STOC/FOCS
- Demonstrates advanced understanding

Implementation:

- Complexity: Medium
- Expected range: $k \leq 30$
- Validation on small instances

Research Directions:

1. **Hybrid Kernelization + IC**
   - Apply reduction rules before IC
   - Potential 2-5x speedup
2. **Adaptive Branching**
   - Learn which vertices to branch first
   - Use graph structure metrics
3. **Parallelization**
   - Independent branches in parallel
   - Multi-core speedup potential
4. **Improved Compression**
   - Smart partition enumeration
   - Structure-based pruning

Algorithm 1: Iterative Compression for FVS

Idea: Build solution incrementally and compress when too large.

Algorithm

Order vertices $v_1, \ldots, v_n$
$F \leftarrow \{v_1\}$
for $i = 2$ to n **do**
    $F \leftarrow F \cup \{v_i\}$
    if $|F| = k + 1$ **then**
        for all partitions (F_1, F_2) of F **do**
            find $Y \subseteq V \setminus F_2$
            if $F_1 \cup Y$ is an FVS **then**
                update F and **break**
            end if
        end for
        if no update performed **then**
            return NO
        end if
    end if
end for
return F **if** $|F| \leq k$

Key Insight: Compress size $k+1$ to k

Time: $O(5^k \cdot n^2)$

Algorithm 2: Kernelization + Bounded Search Tree

Core Concept

Reduce graph size via preprocessing rules, then apply exact search on smaller kernel

Reduction Rules:

- Degree 0, 1, 2 vertices
- Self-loops, duplicate edges
- Crown decomposition
- Flower rule

Search Strategy:

- Smart vertex selection
- Bounded depth: k
- Branching factor: ≤ 4

Research Directions:

1. **New Reduction Rules**
   - Graph-specific rules
   - Theoretical contribution
2. **Rule Application Order**
   - Optimize ordering
   - ML-based prediction
3. **Kernelization Bounds**
   - Tighter bounds for special classes
   - Experimental validation
4. **Smart Branching**
   - Centrality-based selection
   - ML-guided search

Kernelization + Bounded Search Tree

Idea: Reduce the graph, then branch on remaining cycles.

Algorithm

Apply reduction rules:
    remove degree ≤ 1 vertices
    contract degree-2 vertices
    include self-loop vertices
Obtain kernel (G', k')
function Search(G', k')
if G' is acyclic **then**
    return solution
end if
if $k' < 0$ **then**
    return NO
end if
find a cycle C
for each $v \in C$ **do**
    recurse on $(G' - v,\ k' - 1)$
end for

Key Insight: Small kernel $\Rightarrow$ small search tree

Time: $O(4^k + n^2)$

Algorithm 3: Memetic Algorithm (GA + Local Search)

Core Concept

Population evolution + local optimization for large-scale instances

Components:

- Smart initialization (greedy + random)
- FVS-aware crossover operators
- Centrality-based mutation
- Problem-specific local search

Practical Relevance:

- Industry standard
- Handles $n > 1000$ vertices
- Near-optimal (5-10% gap)

Research Directions:

1. **Cycle-Preserving Crossover**
   - Preserve cycle-breaking structures
   - Novel contribution
2. **Hybrid Exact-Metaheuristic**
   - Use IC for small components
   - GA for large core
3. **ML Integration**
   - Predict promising vertices
   - Learn operator selection
4. **Multi-Objective**
   - Minimize size + maximize robustness
   - Pareto frontier

Memetic Algorithm for FVS

Idea: Evolutionary search combined with local optimization.

Algorithm

1: Encode solution as a permutation of vertices
2: Decode by removing vertices until graph is acyclic
3: Initialize population (greedy + random)
4: **for** each generation **do**
5: Evaluate fitness: $-|FVS|$
6: Select parents (tournament)
7: Crossover (cycle-aware)
8: Mutation (swap cycle-heavy vertices)
9: Local search (hill-climbing)
10: **end for**
11: **return** best solution found

Key Insight: Exploration + refinement

Type: Heuristic (near-optimal)

Why This Three-Algorithm Combination?

Aspect	**Iter. Compression**	**Kernel + BST**	**Memetic GA**
Size Range	Small-Medium ($k \leq 30$)	Medium (after kernel)	Large ($n > 1000$)
Paradigm	FPT Dynamic	FPT + Preprocess	Evolutionary
Guarantee	Optimal	Optimal	Approximate
Speed	Medium	Slow (small kernel)	Fast
Research	FPT techniques	Reduction rules	EA design
Practice	Moderate k	Preprocessing insight	Industry standard

Experimental Design

Overview: Comprehensive Evaluation Framework

- **Dataset Construction:** Diverse synthetic and real-world graph instances covering various structural properties and scales
- **Performance Metrics:** Multi-faceted evaluation including solution quality, computational efficiency, and algorithm robustness
- **Experimental Protocol:** Reproducible methodology with controlled conditions, statistical rigor, and fair comparisons
- **Analysis Plan:** Statistical tests, visualization strategies, and interpretation guidelines

Goal

Provide actionable insights into when each algorithm excels and practical guidance for algorithm selection

Dataset Design: Comprehensive Test Suite

Design Rationale

Our dataset covers diverse graph structures to test algorithm behavior under different conditions:

- **Size variation:** Small (verification) $\rightarrow$ Medium (comparison) $\rightarrow$ Large (scalability)
- **Density spectrum:** Sparse to dense graphs (affects FVS size and difficulty)
- **Structural diversity:** Random, structured, scale-free, small-world properties

Dataset Design: Comprehensive Test Suite (Continued)

Graph Type	Sizes/Parameters	Count	Evaluation Purpose
Random (ER)	$n \in \{10, 20, 50, 100, 200, 500, 1000\}$ $p \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$	350	Baseline performance, density sensitivity
Scale-Free (BA)	$n \in \{10, 20, 50, 100, 200, 500, 1000\}$ $m \in \{2, 3, 5\}$	210	Real-world structure, hub influence
Grid	$\{3 \times 3, 5 \times 5, 10 \times 10,$ $20 \times 20, 30 \times 30\}$	25	Structured topology, predictable bounds
Small-World (WS)	$n \in \{20, 50, 100, 200, 500\}$ $k \in \{4, 6, 8\}, \beta \in \{0.1, 0.3, 0.5\}$	450	Clustering effects, local vs global structure
Cycle-Heavy	4 cycle densities $\times$ 3 connection patterns	120	Worst-case analysis, stress testing
Trees	$n \in \{10, 20, 50, 100, 200\}$ Random, Balanced, Star	75	Sanity check (expected FVS $= 0$)
Real-World	Karate, Dolphins, Email, Blogs, Power grid, Bio networks	15-20	Practical validation, domain relevance

Total: $\sim$1,250 synthetic + 20 real-world instances with controlled variation

Performance Metrics: Multi-Dimensional Evaluation

1. Solution Quality Metrics:

1. **FVS Size:** $|S|$ (primary objective)
   - Absolute value and ranking
2. **Approximation Ratio:**

$$\text{Ratio} = \frac{|S_{\text{alg}}|}{|S_{\text{opt}}|}$$

3. **Relative Gap:**

$$\text{Gap} = \frac{|S_{\text{alg}}| - |S_{\text{best}}|}{|S_{\text{best}}|} \times 100\%$$

4. **Correctness:** Verify $G - S$ is acyclic
   - Must achieve 100% validity

2. Computational Efficiency:

1. **Wall-Clock Time** (end-to-end)
2. **CPU Time** (actual computation)
3. **Memory Usage** (peak RAM)
4. **Iterations/Generations** (convergence)

3. Robustness & Consistency:

1. **Coefficient of Variation:**

$$CV = \frac{\sigma_{\text{size}}}{\mu_{\text{size}}} \times 100\%$$

2. **Success Rate:**

$$\text{Success} = \frac{\#\text{ solved}}{\#\text{ attempted}} \times 100\%$$

Experimental Protocol: Ensuring Rigor and Reproducibility

Implementation Environment

Controlled Setup for Fair Comparison:

- **Language:** Python 3.10+ (consistency across implementations)
- **Libraries:** NetworkX (graphs), NumPy (computation), Matplotlib/Seaborn (visualization), Pandas (analysis)
- **Hardware:** [To specify: CPU model, cores, RAM, OS] - document for reproducibility
- **Isolation:** Single-threaded execution, no concurrent processes

Experimental Protocol: Ensuring Rigor and Reproducibility(Continued)

Execution Protocol

Standardized Testing Procedure:

- **Repetitions:** 10 independent runs for randomized algorithms (GA), 1 run for deterministic (IC, Kernel+BST)
- **Timeout Strategy:**
  - Small instances ($n \leq 50$): 60 seconds
  - Medium instances ($50 < n \leq 200$): 300 seconds (5 min)
  - Large instances ($n > 200$): 600 seconds (10 min)
- **Random Seeds:** Fixed seeds (1-10) for reproducibility of stochastic algorithms
- **Warm-up:** Discard first run to eliminate JIT/caching effects
- **Data Logging:** Structured CSV/JSON with all metrics, timestamps, and parameters

Experimental Design: Core Research Questions

Research Questions Driving Our Experiments

1. **RQ1: Solution Quality**
   - How close do our algorithms get to optimal solutions?
   - Which algorithm provides best quality for different graph types?
2. **RQ2: Scalability & Efficiency**
   - What is the largest instance each algorithm can handle?
   - How does runtime scale with graph size and density?
3. **RQ3: Quality-Time Trade-off**
   - Given a time budget, which algorithm is most effective?
   - Where is the sweet spot: exact slow vs. fast approximate?
4. **RQ4: Graph Structure Sensitivity**
   - Do algorithms perform differently on random vs. scale-free graphs?
   - Which structural properties favor which algorithms?
5. **RQ5: Algorithm Stability**
   - How consistent are results across multiple runs (for GA)?
   - What is the variance in solution quality?

Key Experiments: Systematic Evaluation Plan

1. **Experiment 1: Correctness & Validation**
   - *Goal:* Verify all solutions are valid feedback vertex sets
   - *Method:* For each solution S, verify $G - S$ is acyclic using DFS
   - *Expected:* 100% validity across all algorithms and instances
2. **Experiment 2: Solution Quality Comparison**
   - *Goal:* Compare FVS sizes across algorithms
   - *Method:* Box plots per graph type, statistical significance tests (Wilcoxon, Friedman)
   - *Analysis:* Rank algorithms by median FVS size, identify significant differences
3. **Experiment 3: Runtime & Scalability Analysis**
   - *Goal:* Assess computational efficiency and scaling behavior
   - *Method:* Plot runtime vs. n (log-log scale), fit power-law curves
   - *Analysis:* Identify crossover points, determine practical size limits
4. **Experiment 4: Quality-Runtime Pareto Analysis**
   - *Goal:* Find optimal algorithm for given time constraints
   - *Method:* Pareto frontier plot (FVS size vs. time), dominated solution analysis
   - *Insight:* Practical algorithm selection guide

Key Experiments (continued)

5. **Experiment 5: Graph Structure Sensitivity Analysis**
   - *Goal:* Understand how graph properties affect algorithm performance
   - *Method:* Performance heatmap (algorithm $\times$ graph type), correlation analysis
   - *Analysis:* Identify which algorithm excels on which structure (e.g., GA on scale-free, IC on sparse)

6. **Experiment 6: Parameter Sensitivity (Genetic Algorithm)**
   - *Goal:* Optimize GA hyperparameters for best performance
   - *Variables:* Population size $\{20, 50, 100, 200, 500\}$, Mutation rate $\{0.01, 0.05, 0.1, 0.2\}$, Crossover rate $\{0.5, 0.7, 0.8, 0.9\}$
   - *Method:* Grid search or Latin hypercube sampling, response surface analysis
   - *Output:* Recommended parameter settings

7. **Experiment 7: Convergence Analysis**
   - *Goal:* Study how solution quality improves over time/iterations
   - *Method:* Track best FVS size vs. time/generation, convergence curves
   - *Insight:* Determine when to stop (diminishing returns)

8. **Experiment 8: Optimality Gap Assessment**
   - *Goal:* Quantify approximation quality on small instances
   - *Method:* For $n \leq 20$, compute optimal using brute force or exact solver
   - *Metric:* $(|S_{\text{approx}}| - |S_{\text{opt}}|)/|S_{\text{opt}}| \times 100\%$

Additional Experiments & Statistical Analysis

9. **Experiment 9: Real-World Validation**
   - *Goal:* Validate performance on realistic network instances
   - *Datasets:* Social networks, biological networks, infrastructure graphs
   - *Analysis:* Compare synthetic vs. real-world performance, domain-specific insights
10. **Experiment 10: Robustness & Stability Testing**
   - *Goal:* Assess consistency of randomized algorithm (GA)
   - *Method:* Coefficient of variation across 10 runs, best/worst/median analysis
   - *Insight:* Reliability for practical deployment

Statistical Analysis Plan

Ensuring Scientific Rigor:

- **Hypothesis Testing:** Wilcoxon signed-rank test (pairwise), Friedman test (multiple algorithms)
- **Effect Size:** Compute Cohen's d or rank-biserial correlation for practical significance
- **Confidence Intervals:** Report 95% CI for mean/median metrics
- **Multiple Testing Correction:** Bonferroni or Holm-Bonferroni adjustment

Visualization & Reporting Strategy

Planned Visualizations:

1. **Box Plots**
   - FVS size distribution per algorithm
   - Identify outliers and spread
2. **Scaling Curves**
   - Runtime vs. n (log-log)
   - Empirical complexity validation
3. **Pareto Frontiers**
   - Quality vs. time trade-off
   - Dominated region shading
4. **Heatmaps**
   - Performance matrix (algorithm $\times$ graph type)
   - Parameter sensitivity grids
5. **Convergence Plots**
   - Best solution over time/generation
   - Multiple runs with confidence bands

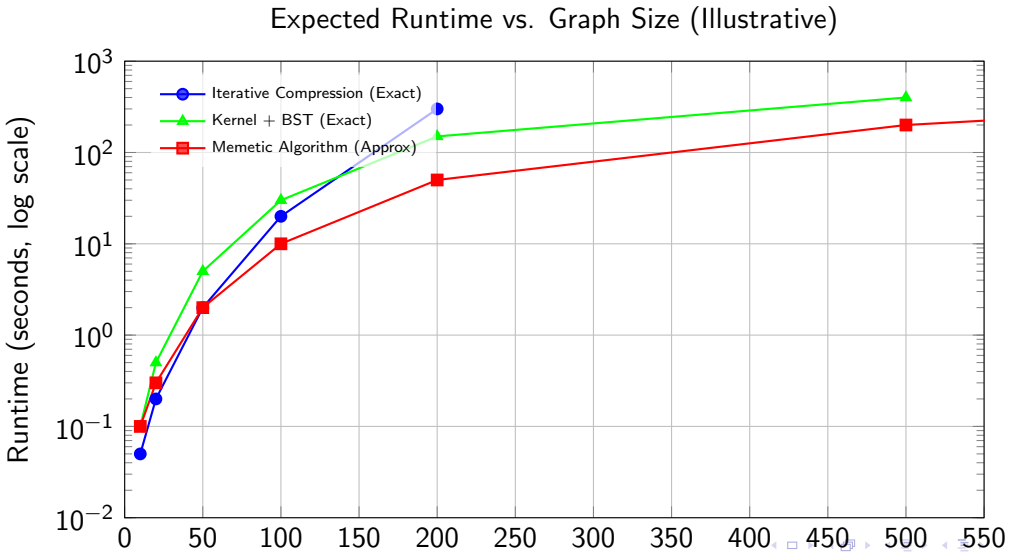
Reporting Structure:

- **Summary Tables:** Mean, median, std dev, min/max for key metrics
- **Performance Profiles:** Cumulative distribution of solution quality
- **Ranking Analysis:** Win/tie/loss records between algorithms
- **Case Studies:** Deep dive into interesting instances

Deliverables

- Comprehensive experimental report
- Interactive visualizations (if time permits)
- Open-source code repository
- Reproducible benchmark suite

Expected Results Visualization



Applications

Overview

- Survey theoretical and practical applications of FVS.
- Provide domain examples: OS deadlocks, VLSI testing, biology, social networks, finance.
- Discuss impact and real-world relevance of solutions.

Application 1 : Deadlock Detection in Operating Systems

Problem

- Processes compete for shared resources
- Circular waiting leads to deadlock
- Goal: resolve deadlock with minimal impact

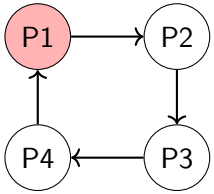
Graph Model

- Vertices represent processes
- Directed edges represent wait-for relations
- Cycles correspond to deadlocks

Role of FVS

- FVS models processes whose removal breaks all cycles

FVS $= \{P1\}$



Deadlock cycle in a wait-for graph

Application 2: VLSI Circuit Testing

Challenge

- Modern chips contain millions of logic gates
- Feedback loops make testing computationally hard
- Acyclic circuits are testable in linear time

FVS-Based Solution

- Model circuit as a graph
- Compute a small FVS to break all cycles
- Fix FVS gates and test remaining acyclic circuit
- Test FVS gates separately

Example

- 32-bit ALU circuit
- 5,000 gates total
- 200 feedback paths
- FVS size: 50 gates

Fixing 50 gates $\Rightarrow$ 4,950 acyclic gates

Benefits

- Reduced test time
- Lower power consumption
- Smaller hardware overhead

Application 3: Gene Regulatory Networks

Biological Motivation

- Genes regulate each other via feedback loops
- Cycles control cell behavior and disease states

Graph Model

- Vertices: genes
- Edges: regulatory interactions
- Cycles: feedback regulation

Role of FVS

- Small set of FVS genes controls all feedback : Interpreted as **master regulators**

Cancer Study Example

- Breast cancer regulatory network
- 500 genes analyzed
- FVS: 12 key genes
- 10 known and 2 novel drug targets

Applications

- Drug target discovery
- Gene knockout analysis
- Personalized medicine

Social and Financial Networks

Social Network Analysis

- Feedback loops amplify influence and information spread
- FVS nodes correspond to key influencers or superspreaders
- Targeting FVS improves marketing efficiency
- Disrupting FVS reduces misinformation propagation

Financial Systemic Risk

- Financial institutions form highly interconnected networks
- Cycles represent circular dependencies and systemic risk
- FVS identifies systemically important institutions
- Used for regulation, monitoring, and crisis prevention

Theoretical Applications

- **Complexity Theory**
  - Used in NP-completeness reductions
  - Benchmark problem for cycle-breaking hardness
- **Algorithm Design**
  - Testbed for FPT(Fixed Parameter Tractable) techniques
  - Motivates iterative compression and branching methods
- **Approximation Theory**
  - Used to study approximation limits
  - Reference problem for constant-factor approximations
- **Graph Structure Analysis**
  - Tool for analyzing treewidth and feedback structure
  - Applied in kernelization and graph decompositions

Potential Research Directions

Overview

- Highlight open theoretical problems (approximation gap, faster FPT, kernels).
- Point out practical directions: ML integration, dynamic and distributed algorithms.